Reflection Template

Use Case	UC-1
Use Case Name	Create User
Requirement IDs	FR1, FR19, FR20, FR21, FR22, FR23
Matching Feature Comparison Sets	CS-01, CS-02, CS-03
Implementation Order	Java first -> Kotlin second

Feature Relevance

Feature/s:

Null safety, data classes/records, type inference

Did the feature/s occur naturally in this/these context/s?

Yes, no features were forced on the implementation. Used data classes natural and in Java implementation records. Used type inference in UserService naturally for both backends.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	4	5
Compactness of logic	4	4
Perceived implementation effort	3	5

Justifications

Readability:

Both good readability through layers, but Kotlin version bit more compact because data classes eliminate boilerplate and constructor injection syntax is more concise. Java records improved readability compared to POJOs but controller and entity still require more effort.

Compactness of logic:

The service logic almost identical, but Java lower even tho records were used. Difference lies in entity, cause Kotlin requires an explicit no-arg constructor for JPA, and the mapper “user.id ?: throw IllegalStateException(...)” is more compact than an explicit null check in Java.

Implementation effort:

Kotlin felt faster to implement due to a bit less boilerplate.

Observed structural differences

With Java records (instead of POJOs), DTOs are in a single line in both languages the compactness advantage of Kotlin data classes is less visible here.

The Kotlin JPA entity requires an explicit no-arg secondary constructor. Java entities do not.

Kotlin enforces non-nullability at compile time.

Final comparative summary

Both backends follow the same structure and are equally easy to trace. With Java records (not POJOs), the DTO boilerplate is mostly gone, so Kotlin's compactness advantage is smaller than expected. The clearest remaining difference is null safety. Kotlin catches potential null issues at compile time, while Java does not. Overall Kotlin still comes out slightly ahead, but not by much for this use case.